

Tıp Disiplininden Yazılım Mühendisliğine Aktarılabilecek İlkeler: Sistematik Bir Taksonomi

ÖFY

July 2026

Tıp mesleği, hem güçlü bir teorik/bilimsel temele hem de tekrar yoluyla kazanılan güçlü bir pratik bilgiye aynı anda sahip nadir disiplinlerden biridir. Bu iki yönlülük, tıbbi hatanın, belirsizliğin ve öznelliğin sistematik biçimde kurum-sallaştırıldığı bir alan haline getirmiştir — yani tıp, sadece “doğru bilgiyi bilme” üzerine değil, aynı zamanda “hata yapma ihtimalini yönetme” üzerine kurulu bir disiplindir. Aşağıdaki taksonomi, bu disiplinden yazılım mühendisliğine aktarılabilecek otuz ilkeyi dokuz kategori altında, her ilkenin hem tıptaki hem de yazılımdaki karşılığı ayrıntılı biçimde açıklanarak sunmaktadır.

Tanı ve Nedensellik Disiplini

Bu kategori, bir sorunun “gerçek nedenini” bulma sürecinin nasıl sistematikleştirildiğiyle ilgilidir.

Diferansiyel Tanı Yöntemi

Bir hasta bir şikayetle geldiğinde (örneğin göğüs ağrısı), hekim tek bir olasılığa hemen atlamaz. Bunun yerine, o şikayeti açıklayabilecek tüm olası hastalıkların bir listesini çıkarır (kalp krizi, mide reflüsü, kas gerilmesi, akciğer pıhtısı gibi) ve bu listeyi, hem olasılığa hem de tehlike derecesine göre sıralayarak, en ucuz/en hızlı testlerle eleyerek daraltır. Yani önce “en tehlikeli ama makul olan” ihtimal dışlanır, sonra sırayla diğerleri. Yazılımda debug etme süreci genellikle bunun tam tersidir: bir hata görüldüğünde geliştirici çoğu zaman rastgele bir yeri değiştirip “bakalım düzelecek mi” der, sistematik bir hipotez listesi çıkarmaz. Diferansiyel tanı disiplinini yazılıma taşımak, bir hatayla karşılaşıldığında önce olası nedenlerin bir listesini çıkarıp, bunları en hızlı test edilebilir olandan başlayarak sistematik şekilde elemek anlamına gelir.

“At mı Zebra mı” İlkesi

Tıp eğitiminde çok bilinen bir söz vardır: “Nal sesi duyduğunda at düşün, zebra değil.” Bunun anlamı: bir semptom görüldüğünde önce yaygın, sıradan, istatistiksel olarak olası açıklamalar düşünülmeli, nadir görülen egzotik hastalıklar (zebra) en son akla gelmelidir. Yazılımda bunun tersi sık görülür: bir hata

çıkıldığında geliştirici genelde önce “framework’te gizli bir bug olabilir” ya da “bu bir eşzamanlılık (race condition) sorunu olabilir” gibi karmaşık/nadir açıklamalara atlar — oysa gerçek sebep çoğunlukla basit bir yazım hatası ya da yanlış bir ayar dosyasıdır. Bu ilkeyi yazılıma taşımak, hata ayıklarken önce en sıradan/basit açıklamaları kontrol etmek, egzotik teorilere ancak basit olasılıklar tükendiğinde geçmek demektir.

Taban Oranı (Base Rate) Düşüncesi

Bu, istatistikte önemli ama sezgiye aykırı bir kavramdır. Örnek: bir hastalığın toplumda görülme sıklığı (prevalansı) çok düşükse, o hastalığı “%99 doğrulukla” tespit eden bir test bile, pozitif çıkan sonuçların çoğunun aslında yanlış alarm olduğu anlamına gelebilir — çünkü hasta olmayan insan sayısı o kadar fazladır ki, testin küçük hata payı bile mutlak sayıda çok fazla yanlış pozitif üretir. Tıp eğitiminde bu mantık (Bayes teoremi temelli düşünme) açıkça öğretilir. Yazılımda alarm/izleme sistemleri kurulurken bu mantık nadiren düşünülür: bir alarmın “doğruluk oranı” yüksek görünse bile, gerçek dünyada kaç tane yanlış pozitif üreteceği ve ekibin zamanla bu alarmları görmezden gelmeye başlayacağı (buna “alarm yorgunluğu” denir) genellikle hesaplanmaz.

Biyopsikososyal Model

Modern tıp bir hastalığı yalnızca biyolojik (vücuttaki fiziksel bozukluk) bir mesele olarak görmez; hastanın psikolojik durumunu (stres, kaygı) ve sosyal koşullarını (aile, iş, ekonomik durum) da aynı anda değerlendiren bir çerçeve kullanır — çünkü aynı semptomun farklı hastalarda tamamen farklı kök nedenleri olabilir. Yazılımda bir performans sorunu ya da kullanıcı şikayeti genellikle sadece teknik açıdan (“kod yavaş çalışıyor”) ele alınır; oysa sorunun asıl kaynağı organizasyonel olabilir (yanlış önceliklendirme yapılmış), süreçsel olabilir (yeterli test yapılmamış) ya da iletişimsel olabilir (kullanıcıya yanlış beklenti verilmiş). Bu ilkeyi yazılıma taşımak, bir sorunu değerlendirirken sadece teknik boyutuna değil, süreç ve iletişim boyutlarına da aynı anda bakmak demektir.

Tumor Board / Multidisipliner Konsey

Kanser gibi karmaşık hastalıklarda tedavi kararı tek bir doktora bırakılmaz; onkolog (kanser uzmanı), radyolog (görüntüleme uzmanı), patolog (doku analizi uzmanı) ve cerrah aynı toplantıda, aynı verilere birlikte bakıp ortak bir karar üretir. Bu, tek bir kişinin kör noktalarının kararı etkilemesini engeller. Yazılımda mimari kararlar (örneğin hangi veritabanının kullanılacağı, güvenlik mimarisinin nasıl kurulacağı) genellikle tek bir “tech lead”in onayıyla geçilir; güvenlik, performans, operasyon ve ürün bakış açılarının aynı toplantıda birlikte değerlendirildiği zorunlu bir süreç nadiren vardır.

Önceliklendirme ve Sürekli İzleme

Bu kategori, “hangi soruna önce bakılmalı” ve “kötüleşme nasıl erken fark edilir” sorularıyla ilgilidir.

Triyaj Mantığı

Acil serviste hastalar sıraya göre değil, aciliyet derecesine göre görülür — kalp krizi geçiren biri, saatler önce gelmiş ama ayak bileği burkulmuş birinden önce muayene edilir. Bu, kaynakların (doktor zamanı) en kritik yere yönlendirilmesini sağlayan objektif bir sıralama sistemidir. Yazılımda iş/hata önceliklendirmesi genellikle “kim en çok şikayet ediyorsa ona bakılır” mantığıyla işler; üretim ortamını çökertebilecek ciddi bir hata ile sadece görsel bir kusur arasında objektif kriterlere dayalı bir ayırım nadiren sistematik biçimde yapılır.

Vital Bulgular / Sürekli İzleme Kültürü

Hastanede yatan bir hastanın nabızı, tansiyonu, kandaki oksijen oranı gibi değerleri sürekli ve otomatik olarak ölçülür. Bu izleme sayesinde, hastanın durumu kötüleşmeye başladığında — hasta henüz kendisi bir şikayette bulunmadan — sistem bunu erken fark edip uyarı verir. Yazılımdaki “gözlemlenebilirlik” (observability/monitoring) kültürü bunun bir versiyonudur, ama genellikle tepkiseldir: sistem çöktükten sonra kayıtlara (loglara) bakılır. Tıptaki mantık daha çok “erken uyarı sistemi” kurmaktır — bir ölçüm normal aralığın dışına çıkar çıkmaz, henüz gözle görülür bir sorun oluşmadan müdahale edilir.

Hata Yönetimi ve Kurumsal Öğrenme

Bu kategori, hataların nasıl önlendiği ve yaşandıktan sonra sistemin bunlardan nasıl öğrendiğiyle ilgilidir.

Checklist Kültürü

Cerrah Atul Gawande’nin araştırması şunu göstermiştir: dünyanın en deneyimli cerrahları bile, ameliyattan önce basit bir kontrol listesini (doğru hasta mı, doğru taraf mı, gerekli ilaç verildi mi) uyguladıklarında komplikasyon oranları belirgin biçimde düşmektedir. Bunun nedeni, deneyimin kendisinin güvenlik garantisi olmamasıdır — yorgunluk, dikkat dağınıklığı, “bunu zaten biliyorum” rahatlığı herkeste hata üretebilir. Yazılımda deploy (yayına alma) öncesi kontrol listeleri ve code review şablonları bu mantığın kısmen uygulanmış hâlidir; ama tıptaki gibi “en kıdemli kişi de aynı listeyi uygular” kültürü nadirdir — çoğu ekipte checklist yalnızca yeni başlayanlar için gerekli görülür.

Morbidity and Mortality (M&M) Konferansları

Hastanelerde düzenli olarak yapılan bu toplantılarda, kötü sonuçlanan vakalar tartışılır ama kimin hata yaptığını bulmak amacıyla değil, sistemin neresinin bu

hataya izin verdiğini anlamak amacıyla — isim vermeden, suçlayıcı olmayan bir dille (“blameless”). Yazılım dünyasındaki “blameless postmortem” (suçlamasız olay sonrası analiz) kültürü doğrudan buradan esinlenmiştir, ancak tıpta bu pratik çok daha eski, çok daha düzenli ve hiyerarşiden bağımsız (en kıdemli doktor da eleştiriye açık) bir ritüel haline gelmiştir.

Otopsi/Nekropsi Kültürü

Bir hasta, özellikle beklenmedik şekilde kaybedildiğinde, otopsi yoluyla gerçekte ne olduğu sistematik olarak araştırılır — en kötü sonuçtan bile bilgi çıkarma zorunluluğu vardır. Yazılımda bir proje tamamen terk edildiğinde ya da bir sistem kapatıldığında, bundan ne öğrenilebileceğine dair bir “post-mortem” (olay sonrası inceleme) nadiren yapılır; başarısız projeler genellikle hiç incelenmeden sessizce unutulur.

Kademeli Müdahale ve Risk Sınırlama

Bu kategori, büyük bir değişikliğin ya da yeni bir kişinin sorumluluğunun aşama aşama, kontrollü biçimde nasıl artırıldığıyla ilgilidir.

Residency/Staj Sistemi

Tıp eğitiminde bir asistan doktor önce sadece izler, sonra deneyimli bir doktora yardım eder, sonra gözetim altında kendisi yapar, ve ancak belirli sayıda vakayı başarıyla tamamladıktan sonra tek başına sorumluluk alabilir — bu geçişlerin her biri açıkça tanımlı, ölçülebilir kriterlere bağlıdır. Yazılımda “junior-mid-senior” geçişleri genellikle çok daha belirsiz ve öznel (“hissiyat”) temellidir; net, ölçülebilir bir yeterlilik çerçevesi nadiren uygulanır.

Faz I–IV Klinik Denemeler

Yeni bir ilaç asla doğrudan geniş bir hasta kitlesine verilmez. Önce çok küçük bir grupta yalnızca güvenliği test edilir (Faz I), sonra biraz daha geniş bir grupta etkinliği ölçülür (Faz II), sonra çok daha büyük, karşılaştırmalı bir çalışmayla kanıtlanır (Faz III), ve piyasaya çıktıktan sonra bile uzun süreli izleme devam eder (Faz IV). Yazılımdaki “canary release” (yeni bir özelliği önce küçük bir kullanıcı grubuna açma) kültürü bu mantığın bir versiyonudur, ama disiplin çok daha gevşektir — çoğu ekip test ortamında sorun görmeyince doğrudan tüm kullanıcılara açar; tıptaki gibi her aşamada durup değerlendirme titizliği nadiren uygulanır.

Karantina/Enfeksiyon Kontrolü

Hastanelerde bulaşıcı bir hastanın fiziksel olarak izole edilmesi, el yıkama zorunluluğu gibi protokoller, bir sorunun tek bir noktadan tüm hastaneye yayılmasını mimari/yapısal olarak imkansız kılar. Yazılımdaki karşılığı “blast radius” (bir arızanın etki alanını sınırlama) kavramıdır — bir servisin çökmesinin diğer

servisleri etkilememesi için kullanılan devre kesiciler (circuit breaker) gibi mekanizmalar. Ancak bu mekanizmalar genellikle bir felaket yaşandıktan *sonra* eklenir; tıptaki gibi baştan itibaren zorunlu bir tasarım ilkesi olarak nadiren uygulanır.

İletişim, Bağlam Aktarımı ve Şeffaflık

Bu kategori, bilginin bir kişiden diğerine kayıpsız ve dürüst biçimde nasıl aktarıldığıyla ilgilidir.

Informed Consent ve Risk İletişimi

Bir doktor bir prosedürün risklerini hastaya anlaşılır biçimde anlatmak zorundadır — belirsizliği gizlemek değil, açıkça iletmek etik bir zorunluluktur. Yazılımda kullanıcıya karşı bu netlik genellikle zayıftır: bir güncellemenin taşıdığı riskler, bir özelliğin sınırları, bir sistemin ne zaman güvenilmez hale gelebileceği nadiren açıkça iletilir; genellikle “her şey sorunsuz çalışacak” izlenimi verilir.

Anamnez Disiplini

İyi bir doktor, hastanın söylediği ilk cümleyi (“karnım ağrıyor”) tek bilgi kaynağı olarak kabul etmez; ne zaman başladığını, neyin artırıp azalttığını, geçmişte benzer bir durum olup olmadığını sistematik olarak sorar. Yazılımda bir hata raporu genellikle kullanıcının ilk cümlesiyle (“uygulama açılmıyor”) ele alınır; hangi adımların izlendiği, ne zamandır sorunun sürdüğü, hangi ortamda oluştuğu gibi standart bir bilgi toplama şablonu nadiren zorunlu kılınır.

Prognoz İletişimi

Doktor “bu tedavi işe yarayabilir” gibi belirsiz bir ifade yerine, mümkün olduğunca “bu tedavinin başarı oranı yüzde 70, yan etki riski yüzde 5” gibi somut olasılıklarla konuşur. Yazılımda proje süresi tahminleri genellikle tek bir sayı olarak verilir (“iki hafta sürer”), oysa gerçek durum neredeyse her zaman bir olasılık dağılımıdır (en iyi durumda, en kötü durumda, en olası durumda). Bu ilkeyi yazılıma taşımak, tahminleri tek sayı yerine olasılıklarla ifade etmek demektir.

SBAR / Devir Teslim Protokolü

Bir hastanın bakımı bir doktordan diğerine (vardiya değişiminde) devredilirken, bilgi standart bir formatla (Durum – Arka Plan – Değerlendirme – Öneri, İngilizce kısaltmasıyla SBAR) aktarılır; bu, kritik bir bilginin unutulmasını yapısal olarak engeller. Yazılımda bir işin bir geliştiriciden diğerine devri (biri izne çıkarken ya da işten ayrılırken) genellikle standart bir format izlemez; “şurada bir doküman var, bakarsın” düzeyinde kalır ve bağlamın büyük kısmı kişiyle birlikte kaybolur.

İlaç Uzlaştırması

Bir hasta bir bölümden diğerine (örneğin ameliyathaneden yoğun bakıma) geçtiğinde, kullandığı tüm ilaçların listesi sıfırdan, tek tek kontrol edilir — çünkü bu tür geçişlerde bir ilacın unutulması ya da iki ilacın birbiriyle çakışması en sık görülen hata kaynağıdır. Yazılımda bir servisin ayarlarının ve bağımlılıklarının bir ortamdan diğerine (test ortamından canlı ortama) geçerken tam bir kontrolü nadiren bu titizlikte yapılır; genellikle “muhtemelen aynıdır” varsayımıyla geçilir ve canlı ortama özel bir ayarın unutulması klasik bir kesinti nedenidir.

Grand Rounds

Bu, bir vakanın tüm bölüme (asistanlardan en kıdemli profesöre kadar) açık biçimde sunulup, doktorun tanı sürecindeki her adımın (“neden önce bunu düşündün, neden şunu elemedin”) sorgulandığı bir ritüeldir. Yazılımda code review (kod incelemesi) buna benzer ama genellikle sonucu (yazılan kodu) değerlendirir; kararın arkasındaki düşünme sürecinin kendisi (hangi alternatifler değerlendirildi, neden elendi) aynı açıklıkla nadiren sorgulanır.

Hesap Verebilirlik ve Belgeleme

Bu kategori, kararların neden verildiğinin nasıl kayıt altına alındığıyla ilgilidir.

Kontrendikasyon Düşüncesi

Tıpta bir ilacın ne işe yaradığı kadar, hangi durumlarda kesinlikle verilmemesi gerektiği (kontrendikasyon) de eğitimin ayrı, sistematik ve ezberlenmesi zorunlu bir parçasıdır. Yazılımda “bu yöntemi şu durumda kullanma” bilgisi genellikle dokümanite edilmez; sadece sözlü kültürde (“o zaman bunu denemiştik, sistem çökmüştü”) yaşar ve kişiyle birlikte kaybolur.

Malpractice/Sorumluluk Çerçevesi

Tıpta bir hatanın hukuki ve mesleki sonuçları olabilir; bu, doktorları bir tehdit kültürüyle korkutmaktan çok, her kararın gerekçesinin dosyaya yazılmasını zorunlu kılan bir alışkanlık yaratır. Yazılımda büyük teknik kararların (bir mimari seçimi, bir güvenlik açığının bilinçli olarak ertelenmesi) gerekçesi genellikle hiçbir yerde yazılı olarak kayıt altına alınmaz; birkaç yıl sonra “bunu neden böyle yapmıştık” sorusuna kimse cevap veremez.

İkinci Görüş (Second Opinion) Kurumsallaşması

Ciddi bir tanı ya da tedavi kararında ikinci bir doktorun görüşünü almak hem yaygın hem de meşru bir pratiktir; bu, ilk doktorun yetersiz olduğu anlamına gelmez. Yazılımda code review bunun bir versiyonudur ama genellikle “hızlıca onay al, devam et” kültürüne dönüşmüştür; özellikle mimari ve güvenlik kararlarında bağımsız bir ikinci değerlendirme, hız baskısı yüzünden lüks gibi görülür.

Informed Refusal

Hasta önerilen bir tedaviyi reddedebilir ve bu red, suçlama yapılmadan resmi olarak belgelenir. Yazılımda bir paydaşın önerilen bir teknik değişikliği reddetme hakkı ve bu reddin gerekçesi nadiren bu kadar resmi biçimde kayıt altına alınır; genellikle sözlü bir tartışmayla geçiştirilir.

Kanıtı Dayalılık ve Öznelliğe Şüphecilik

Bu kategori, “bunun işe yaradığını nasıl biliyoruz” sorusuna verilen cevabın kalitesiyle ilgilidir.

Kanıtı Dayalı Pratik (Evidence-Based Medicine)

1990’lardan itibaren güçlenen bu hareket, doktorların “ben böyle öğrendim, böyle yapıyorum” tarzı bir alışkanlık temelinde değil, sistematik derlemeler ve randomize kontrollü çalışmalarla (bir tedavinin gerçekten işe yarayıp yaramadığını ölçen deneyler) karar vermesini zorunlu kılmıştır. Yazılımda “best practice” (en iyi uygulama) denen şeylerin çoğu hiçbir zaman ampirik olarak test edilmemiştir; sadece popüler ya da prestijli kişilerin söylemiyle yayılmış inançlardır.

Placebo/Nocebo Farkındalığı

Tıp, kendi gözleminin bile yanılabilirliğini kabul eder — hastanın “iyileşiyorum” hissi ya da doktorun beklentisi, tedavinin gerçek etkisini algılamayı bozabilir. Bu yüzden çift kör çalışmalar (ne hastanın ne de doktorun kimin gerçek ilacı, kimin sahte ilacı aldığını bilmediği deneyler) yapılır. Yazılımda “bu değişiklik performansı iyileştirdi” gibi iddialar çoğu zaman gerçek bir karşılaştırma testi (A/B testi) ya da kontrol grubu olmadan, sadece “daha hızlı hissediyorum” izlenimiyle kabul edilir.

Sürdürülebilirlik, Olgunluk ve Sınırları Kabul

Bu kategori, “her zaman daha fazla müdahale etmenin doğru olmadığını” kabul etme yeteneğiyle ilgilidir.

Lisans/Sürekli Eğitim Zorunluluğu

Doktorlar belirli aralıklarla yeniden sertifikalanmak ve sürekli eğitim kredisi almak zorundadır çünkü tıp bilgisi hızla değişir ve bunun kurumsal olarak zorunlu kılınması gerektiği kabul edilir. Yazılımda bu tamamen bireysel inisiyatifle bırakılmıştır; hiçbir zorunlu yeniden değerlendirme yoktur, bu da bilgisi yıllar önce donmuş kişilerin sistemde kalabilmesine yol açabilir.

Iatrogenik Zarar Kavramı

“İyileştirmeye çalışırken zarar verme” ihtimali (iatrojenenez) tıpta merkezi bir kavramdır — her müdahale, faydası kadar taşıdığı riskle de tartılır ve bazen “hiçbir şey yapmamak” en doğru tıbbi karardır. Yazılım kültüründe “bir şey yap, düzelt, iyileştir” refleksi genellikle varsayılan olarak iyi niyetli sayılır; “bu değişikliği yapmamak” nadiren ciddi bir seçenek olarak değerlendirilir — oysa üretim ortamındaki birçok kesinti, tam olarak gereksiz bir “iyileştirme” müdahalesinden kaynaklanır.

Palyatif Düşünce

Tıp, her hastalığın tam olarak düzeltilemeyeceğini, bazen amacın iyileştirmek değil acıyı azaltmak ve yaşam kalitesini korumak olduğunu kabul eden gelişmiş bir yaklaşıma (palyatif bakım) sahiptir. Yazılım kültüründe “bu sistem artık düzeltilemez, sadece idare edilebilir” kabulü nadiren olgunlukla yapılır; genellikle ya sonsuz bir düzeltme çabasına (teknik borç sarmalına) girilir ya da sistem tamamen terk edilir — ara bir “kontrollü idare et, yavaşça sonlandır” stratejisi kültürel olarak az gelişmiştir.

Deprescribing

Modern tıpta giderek güçlenen bir pratik, bir hastanın uzun süredir kullandığı ilaçları düzenli olarak gözden geçirip artık gerekli olmayanları bilinçli biçimde kesmektir (“bir zamanlar gerekliydi ama artık değil” kabulü). Yazılımın karşılığı olması gereken şey — kullanılmayan özellik anahtarlarının (feature flag), ölü kod yollarının, artık gerekmeyen entegrasyonların düzenli temizliği — genellikle sistematik bir ritüel değildir; “bir gün temizleriz” denip yıllarca birikir, çünkü bir şey eklemek her zaman bir şeyi çıkarmaktan daha kolay ve daha az riskli görünür.

Uyarıcı Karşı-Örnek

Defansif Tıp

Burada dengeleyici bir nokta gereklidir: tıbbın her özelliği taklit edilmeye değmez. “Defansif tıp”, doktorların dava edilme riskinden korkarak gereksiz test ya da prosedür istemesi anlamına gelir — bu, sistemin ürettiği ciddi bir bozukluktur (patoloji) ve hastaya fayda değil, gereksiz maliyet ve risk ekler. Yazılım dünyasının “her ihtimale karşı aşırı test yazma” ya da “sorumluluktan kaçmak için gereksiz onay zincirleri kurma” eğilimi bunun bir karşılığıdır. Yani tıbbın disiplininden ders alınırken, tıbbın kendi aşırı temkinli bozukluğunun da miras alınmaması gerekir.

Genel Değerlendirme

Tıbbın yazılım mühendisliğine üstünlüğü teknik zekâda değil, hatanın, belirsizliğin ve öznelliğin kurumsallaştırılmış, ritüelleşmiş ve hiyerarşiden bağımsız

biçimde ele alınmasındadır. Yazılım dünyası bu ilkelerin çoğunu “iyi pratik” olarak tanır, ancak bunları tıptaki gibi zorunlu ve sorgulanamaz bir ritüel haline getirmemiştir; disiplin hâlâ büyük ölçüde bireysel tercih ve kişisel deneyim kültürüne bağlı kalmıştır.